# Module 2 & 3 Project — ZIP Contents

## module_2_3/pipeline.py

```
"""Extraction, normalization, validation, and batch output pipeline."""
import json
import logging
from pathlib import Path
from cleaning.text_cleaner import clean_text
from cleaning.date_cleaner import normalize_date
from cleaning.location_cleaner import normalize_location, normalize_mode
from cleaning.prize_cleaner import normalize_prize
from cleaning.url_cleaner import normalize_url
from parsers.generic_html_parser import GenericHtmlParser
from parsers.json_parser import JsonParser
from schema.canonical_schema import canonical_event
from validation.validator import validate_event

logger = logging.getLogger(__name__)

def _normalize_tags(value):
if value is None:
return []
values = value if isinstance(value, list) else re_split_tags(value)
output, seen = [], set()
for item in values:
tag = clean_text(item)
if tag and tag.casefold() not in seen:
output.append(tag)
seen.add(tag.casefold())
return output

def re_split_tags(value):
import re
return re.split(r"[,;|]", str(value))

def normalize_record(record, source_id=None, source_name=None, today_value=None):
if record.get("_parse_error"):
return {"event": None, "status": "invalid", "errors": [record["_parse_error"]], "raw": record}
event_source_id = record.get("source_id") or source_id
event = canonical_event(
title=clean_text(record.get("title")),
organizer=clean_text(record.get("organizer")) or None,
description=clean_text(record.get("description"), remove_emoji=False) or None,
start_date=normalize_date(record.get("start_date")),
end_date=normalize_date(record.get("end_date")),
location=normalize_location(record.get("location")),
mode=normalize_mode(record.get("mode")),
registration_link=normalize_url(record.get("registration_link")),
prize=normalize_prize(record.get("prize")),
tags=_normalize_tags(record.get("tags")),
source_id=str(event_source_id).strip() if event_source_id is not None else None,
source_name=clean_text(source_name) or None,
status=None,
)
status, errors = validate_event(event, today_value=today_value)
event["status"] = status
return {"event": event, "status": status, "errors": errors, "raw": record}

def process_html(html, source_id=None, source_name=None, today_value=None, parser=None):
parser = parser or GenericHtmlParser()
records = parser.parse(html)
return process_records(records, source_id, source_name, today_value)

def process_json(data, source_id=None, source_name=None, today_value=None, parser=None):
parser = parser or JsonParser()
records = parser.parse(data)
return process_records(records, source_id, source_name, today_value)

def process_records(records, source_id=None, source_name=None, today_value=None):
results = {"valid": [], "expired": [], "invalid": [], "summary": {}}
for record in records:
try:
result = normalize_record(record, source_id, source_name, today_value)
if result["status"] == "invalid":
results["invalid"].append({"event": result["event"], "errors": result["errors"], "raw":
result["raw"]})
elif result["status"] == "expired":
results["expired"].append(result["event"])
else:
```

```
results["valid"].append(result["event"])
except Exception as exc:
logger.exception("Unexpected error while processing event record")
results["invalid"].append({"event": None, "errors": [f"Processing error: {type(exc).__name__}:
{exc}"], "raw": record})
results["summary"] = {
"total": sum(len(results[k]) for k in ("valid", "expired", "invalid")),
"valid": len(results["valid"]),
"expired": len(results["expired"]),
"invalid": len(results["invalid"]),
}
return results

def process_file(input_path, output_dir="output", source_id=None, source_name=None):
input_path = Path(input_path)
output_dir = Path(output_dir)
output_dir.mkdir(parents=True, exist_ok=True)
raw = input_path.read_text(encoding="utf-8")
if input_path.suffix.lower() in {".html", ".htm"}:
results = process_html(raw, source_id, source_name or input_path.stem)
elif input_path.suffix.lower() == ".json":
results = process_json(raw, source_id, source_name or input_path.stem)
else:
raise ValueError("Unsupported input type. Use .html, .htm, or .json.")
for category in ("valid", "expired", "invalid"):
(output_dir / f"{category}_events.json").write_text(
json.dumps(results[category], ensure_ascii=False, indent=2), encoding="utf-8"
)
(output_dir / "summary.json").write_text(json.dumps(results["summary"], indent=2),
encoding="utf-8")
return results
```

## module_2_3/README.md

````
# Module 2.3 — Canonical Data Extraction, Cleaning & Validation

Python pipeline that converts raw event HTML/JSON into a canonical event schema for downstream
deduplication.

## Pipeline

```text
Raw HTML / JSON (Himani)
|
v
Source parser -> text/date/location/prize/URL normalization
|
v
Validation -> valid / expired / invalid
|
v
Canonical JSON output (handoff to Sai Varshini)
```

## Requirements

- Python 3.11+
- pip

## Setup (Windows PowerShell)

```powershell
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PowerShell blocks activation for this terminal, run:
```powershell
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\.venv\Scripts\Activate.ps1
```

## Run examples

Process JSON:
```powershell
python main.py examples/sample_input.json --output-dir output --source-name ExampleSource
```
````

Process HTML:
```powershell
python main.py examples/sample_input.html --output-dir output --source-id sample_html_001
--source-name ExampleHTML
```

Outputs:
- `output/valid_events.json`
- `output/expired_events.json`
- `output/invalid_events.json`
- `output/summary.json`

Run tests:
```powershell
python -m pytest -q
```

## Canonical schema

Every normalized event has these keys:

| Field | Type | Notes |
|---|---|---|
| `title` | string | Required; blank titles are invalid |
| `organizer` | string or null | Organizer/host |
| `description` | string or null | Plain text |
| `start_date` | string or null | ISO `YYYY-MM-DD` |
| `end_date` | string or null | ISO `YYYY-MM-DD` |
| `location` | string or null | Known aliases normalized |
| `mode` | string or null | `online`, `offline`, `hybrid`, or preserved unknown |
| `registration_link` | string or null | Valid HTTP(S) URL or null |
| `prize` | number or null | Numeric amount; currency assumed INR when source uses INR/■ |
| `tags` | array of strings | Deduplicated case-insensitively |
| `source_id` | string or null | Stable ID from source preferred |
| `source_name` | string or null | Name of source site |
| `status` | string | `upcoming`, `expired`, or `invalid` |

Invalid records are placed in `invalid_events.json` with `errors` and `raw` fields. Valid and expired outputs contain canonical event objects. The title is the only required field by default, but a stable `source_id` is also required for reliable downstream traceability. The registration URL is optional; if supplied but malformed, the record is rejected. Adjust `validation/validator.py` to match the team's final contract.

## Integration contract

**From Himani:** provide either UTF-8 HTML strings/files or JSON event objects/lists. Preserve source event IDs when possible and tell Shivam which source website generated each record.

**To Sai Varshini:** consume `output/valid_events.json` and `output/expired_events.json` as lists of canonical event objects. Invalid records are isolated in `output/invalid_events.json` and should not enter deduplication unless the team explicitly decides otherwise.

Example programmatic use:
```python
from pipeline import process_json

result = process_json(
[{"title": "Demo", "id": "site_123", "start_date": "2026-11-20"}],
source_name="ExampleSource",
)
canonical_records = result["valid"]
```

## Design notes and limitations

- Generic HTML extraction is best-effort. Each real website usually needs its own parser and selectors; the included `ExampleSourceParser` is a template, not a parser for a real production website.
- JSON key aliases cover common field names, but the team's actual source formats should be added explicitly.
- Ambiguous numeric dates use day-first interpretation (`12-01-2026` means 12 January 2026).
- Known location aliases are configurable in `config.py`; unknown locations are preserved rather than guessed.
- Prize values are numeric amounts. Currency is not represented separately in schema v1.0; the team should confirm whether all prize amounts should be treated as INR.
- A source ID is not a global event ID. Sai Varshini's module should use multiple normalized fields for deduplication rather than relying on `source_id` alone.
- No external website is fetched by this module. It processes supplied HTML/JSON only.

## GitHub

```powershell
git init
git add .
```

```
git commit -m "Build Module 2.3 event extraction and validation"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git
git push -u origin main
```

Replace the remote URL with your repository URL. If the repository already exists and has commits, inspect its history before pushing; do not overwrite teammates' work.

## module_2_3/config.py

```
"""Configuration for Module 2.3."""
from datetime import date

CANONICAL_FIELDS = (
"title", "organizer", "description", "start_date", "end_date",
"location", "mode", "registration_link", "prize", "tags",
"source_id", "source_name", "status",
)

# Only known aliases are normalized. Unknown values are preserved after cleaning.
LOCATION_ALIASES = {
"bangalore": "Bengaluru",
"blr": "Bengaluru",
"bengaluru": "Bengaluru",
"bombay": "Mumbai",
"mumbai": "Mumbai",
"madras": "Chennai",
"chennai": "Chennai",
"calcutta": "Kolkata",
"kolkata": "Kolkata",
"new delhi": "New Delhi",
"delhi": "New Delhi",
}

MODE_ALIASES = {
"online": "online", "virtual": "online", "remote": "online",
"offline": "offline", "in-person": "offline", "in person": "offline",
"onsite": "offline", "on-site": "offline",
"hybrid": "hybrid", "online/offline": "hybrid",
}

def today():
"""Return the local current date; replace in tests for deterministic results."""
return date.today()
```

## module_2_3/main.py

```
"""Command-line entry point."""
import argparse
import json
import logging
from pipeline import process_file

def main():
parser = argparse.ArgumentParser(description="Module 2.3 event extraction, cleaning, and
validation")
parser.add_argument("input", help="Path to an .html, .htm, or .json input file")
parser.add_argument("--output-dir", default="output", help="Directory for JSON output files")
parser.add_argument("--source-id", default=None, help="Fallback source identifier for a single
event")
parser.add_argument("--source-name", default=None, help="Source website/name")
parser.add_argument("--verbose", action="store_true", help="Enable debug logging")
args = parser.parse_args()
logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO, format="%(levelname)s:
%(message)s")
results = process_file(args.input, args.output_dir, args.source_id, args.source_name)
print(json.dumps(results["summary"], indent=2))
print(f"Results written to: {args.output_dir}")

if __name__ == "__main__":
main()
```

## module_2_3/.gitignore

File type: unknown — contents listed in the archive but not rendered in this PDF.

## module_2_3/requirements.txt

```
beautifulsoup4>=4.12.3
python-dateutil>=2.9.0
pytest>=8.0.0
```

## module_2_3/__pycache__/pipeline.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/__pycache__/config.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/tests/test_validation.py

```
from datetime import date
from validation.validator import validate_event

def base_event():
return {"title": "Demo", "source_id": "source_1", "start_date": "2026-11-01",
"end_date": "2026-11-02", "registration_link": "https://example.com/register"}

def test_upcoming():
status, errors = validate_event(base_event(), today_value=date(2026, 10, 13))
assert status == "upcoming" and errors == []

def test_expired():
event = base_event()
event["start_date"] = "2026-01-01"
status, errors = validate_event(event, today_value=date(2026, 10, 13))
assert status == "expired" and errors == []

def test_missing_title_invalid():
event = base_event()
event["title"] = ""
status, errors = validate_event(event, today_value=date(2026, 10, 13))
assert status == "invalid"
assert any("title" in error for error in errors)

def test_bad_date_order():
event = base_event()
event["start_date"], event["end_date"] = "2026-12-01", "2026-11-01"
status, errors = validate_event(event, today_value=date(2026, 10, 13))
assert status == "invalid"
assert any("start_date" in error for error in errors)
```

## module_2_3/tests/test_parsers.py

```
from parsers.json_parser import JsonParser
from parsers.generic_html_parser import GenericHtmlParser

def test_json_aliases():
result = JsonParser().parse({"name": "Demo Event", "id": "x1", "startDate": "2026-11-01"})
assert result[0]["title"] == "Demo Event"
assert result[0]["source_id"] == "x1"

def test_html_basic():
html = '<html><head><meta name="description" content="A demo"></head><body><h1>Demo</h1><a
href="https://example.com/register">Register</a></body></html>'
result = GenericHtmlParser().parse(html)[0]
assert result["title"] == "Demo"
assert result["registration_link"] == "https://example.com/register"

def test_jsonld():
html = '<script type="application/ld+json">{"@type":"Event","name":"JSON-LD
Demo","startDate":"2026-11-01","identifier":"x2"}</script>'
result = GenericHtmlParser().parse(html)[0]
assert result["title"] == "JSON-LD Demo"
```

## module_2_3/tests/test_pipeline.py

```
from datetime import date
from pipeline import process_json

def test_pipeline_categories():
data = [
{"title": "Future", "id": "a1", "start_date": "2026-11-01", "registration_link":
"https://example.com/r", "prize": "INR 50k"},
{"title": "Past", "id": "a2", "start_date": "2026-01-01"},
{"title": "", "id": "a3", "start_date": "2026-11-01"},
]
result = process_json(data, source_name="test", today_value=date(2026, 10, 13))
assert result["summary"] == {"total": 3, "valid": 1, "expired": 1, "invalid": 1}
assert result["valid"][0]["prize"] == 50000
```

## module_2_3/tests/test_cleaning.py

```
from cleaning.text_cleaner import clean_text
from cleaning.date_cleaner import normalize_date
from cleaning.location_cleaner import normalize_location, normalize_mode
from cleaning.prize_cleaner import normalize_prize
from cleaning.url_cleaner import normalize_url

def test_clean_text():
assert clean_text(" <b>Hello</b> world ■ ") == "Hello world"

def test_dates():
assert normalize_date("12th Jan 2026") == "2026-01-12"
assert normalize_date("Jan 12, 2026") == "2026-01-12"
assert normalize_date("12-01-2026") == "2026-01-12"
assert normalize_date("not a date") is None

def test_locations_modes():
assert normalize_location("BLR") == "Bengaluru"
assert normalize_location("Unknown City") == "Unknown City"
assert normalize_mode("Virtual") == "online"

def test_prizes():
assert normalize_prize("INR 50k") == 50000
assert normalize_prize("■50,000") == 50000
assert normalize_prize("50000 Rs") == 50000
assert normalize_prize("undisclosed") is None

def test_urls():
assert normalize_url("https://example.com/register") == "https://example.com/register"
assert normalize_url("javascript:alert(1)") is None
```

## module_2_3/cleaning/location_cleaner.py

```
"""Location and event-mode normalization."""
from config import LOCATION_ALIASES, MODE_ALIASES
from cleaning.text_cleaner import clean_text

def normalize_location(value):
text = clean_text(value)
if not text:
return None
return LOCATION_ALIASES.get(text.casefold(), text)

def normalize_mode(value):
text = clean_text(value)
if not text:
return None
return MODE_ALIASES.get(text.casefold(), text.casefold())
```

## module_2_3/cleaning/url_cleaner.py

```
"""URL normalization and validation."""
from urllib.parse import urlparse, urlunparse
from cleaning.text_cleaner import clean_text

def normalize_url(value):
"""Return normalized HTTP(S) URL or None if invalid."""
text = clean_text(value, remove_emoji=False)
if not text:
return None
if text.startswith("//"):
text = "https:" + text
parsed = urlparse(text)
```

```
if parsed.scheme.lower() not in {"http", "https"} or not parsed.hostname:
return None
if any(ch.isspace() for ch in text):
return None
try:
_ = parsed.port
except ValueError:
return None
return urlunparse((parsed.scheme.lower(), parsed.netloc, parsed.path or "/", parsed.params,
parsed.query, parsed.fragment))
```

## module_2_3/cleaning/text_cleaner.py

```
"""Text normalization helpers."""
import re
import unicodedata
from bs4 import BeautifulSoup

_EMOJI_RE = re.compile(
"["
"\U0001F300-\U0001FAFF"
"\U00002700-\U000027BF"
"\U00002600-\U000026FF"
"]+", flags=re.UNICODE
)
_CONTROL_RE = re.compile(r"[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]")

def clean_text(value, remove_emoji=True):
"""Convert a value to clean plain text without destroying meaningful content."""
if value is None:
return ""
if not isinstance(value, str):
value = str(value)
if "<" in value and ">" in value:
text = BeautifulSoup(value, "html.parser").get_text(" ", strip=True)
else:
text = value
text = unicodedata.normalize("NFKC", text)
text = _CONTROL_RE.sub("", text)
if remove_emoji:
text = _EMOJI_RE.sub("", text)
return re.sub(r"\s+", " ", text).strip()
```

## module_2_3/cleaning/date_cleaner.py

```
"""Date parsing and ISO date normalization."""
import re
from datetime import date, datetime
from dateutil import parser as date_parser

_ORDINAL_RE = re.compile(r"(?i)(\d+)(st|nd|rd|th)\b")

def normalize_date(value, dayfirst=True):
"""Return YYYY-MM-DD or None. Ambiguous numeric dates use dayfirst=True."""
if value is None or str(value).strip() == "":
return None
if isinstance(value, datetime):
return value.date().isoformat()
if isinstance(value, date):
return value.isoformat()
text = str(value).strip()
try:
# Preserve unambiguous ISO dates before applying day-first parsing.
return date.fromisoformat(text[:10]).isoformat()
except (ValueError, TypeError):
pass
text = _ORDINAL_RE.sub(r"\1", text)
try:
parsed = date_parser.parse(text, dayfirst=dayfirst, fuzzy=False)
return parsed.date().isoformat()
except (ValueError, OverflowError, TypeError):
return None
```

## module_2_3/cleaning/prize_cleaner.py

```
"""Prize parsing. Monetary amounts become numeric values; unknowns become None."""
import re
```

```
from cleaning.text_cleaner import clean_text

_MULTIPLIERS = {"k": 1_000, "thousand": 1_000, "l": 100_000, "lakh": 100_000,
"lac": 100_000, "m": 1_000_000, "million": 1_000_000,
"cr": 10_000_000, "crore": 10_000_000}

def normalize_prize(value):
"""Parse common INR prize strings into int/float, otherwise return None."""
if value is None or isinstance(value, bool):
return None
if isinstance(value, (int, float)):
return value if value >= 0 else None
text = clean_text(value).casefold()
if not text or any(term in text for term in ("undisclosed", "not disclosed", "no prize", "n/a",
"none")):
return None
text = text.replace(",", "").replace("■", "").replace("inr", "").replace("rs.", "").replace("rs",
"").strip()
match = re.search(r"(-?\d+(?:\.\d+)?)\s*(crore|cr|million|m|lakh|lac|l|thousand|k)?", text)
if not match:
return None
try:
number = float(match.group(1))
if number < 0:
return None
suffix = match.group(2)
amount = number * _MULTIPLIERS.get(suffix, 1)
return int(amount) if amount.is_integer() else amount
except (ValueError, OverflowError):
return None
```

## module_2_3/cleaning/__init__.py

```
(Empty file)
```

## module_2_3/examples/sample_input.html

```
<!doctype html>
<html>
<head><title>Sample Hackathon 2026</title><meta name="description" content="A student coding
hackathon."></head>
<body>
<h1>Sample Hackathon 2026 ■</h1>
<p>Organizer: Example Foundation</p>
<p>Start Date: 20 Nov 2026</p>
<p>End Date: 22 Nov 2026</p>
<p>Location: BLR</p>
<p>Mode: Hybrid</p>
<p>Prize: INR 50k</p>
<a href="https://example.com/register">Register Now</a>
</body>
</html>
```

## module_2_3/examples/sample_input.json

```
{
"events": [
{
"title": "Future Python Challenge",
"organizer": "Example Tech",
"description": "<p>A friendly coding challenge.</p>",
"start_date": "20 Nov 2026",
"end_date": "22 Nov 2026",
"location": "Bangalore",
"mode": "virtual",
"registration_link": "https://example.com/python",
"prize": "INR 50k",
"tags": ["Python", "Coding", "Python"],
"id": "example_001"
},
{
"title": "Past Design Sprint",
"organizer": "Example Studio",
"start_date": "10 Jan 2026",
"end_date": "11 Jan 2026",
"location": "Mumbai",
"mode": "offline",
```

```
"registration_link": "https://example.com/design",
"prize": "■25,000",
"tags": "design, product",
"id": "example_002"
},
{
"title": "",
"id": "example_invalid"
}
]
}
```

## module_2_3/schema/canonical_schema.py

```
"""Canonical event schema and output helpers."""
from datetime import date
from typing import Any

CANONICAL_FIELDS = [
"title", "organizer", "description", "start_date", "end_date",
"location", "mode", "registration_link", "prize", "tags",
"source_id", "source_name", "status",
]

def canonical_event(**kwargs) -> dict[str, Any]:
"""Return a canonical record with stable field order and predictable nulls."""
event = {field: kwargs.get(field) for field in CANONICAL_FIELDS}
event["title"] = event.get("title") or ""
event["description"] = event.get("description") or ""
event["tags"] = event.get("tags") or []
event["status"] = event.get("status") or "upcoming"
return event

def schema_version():
return "1.0"
```

## module_2_3/schema/__init__.py

```
(Empty file)
```

## module_2_3/validation/validator.py

```
"""Canonical event validation."""
from datetime import date
from urllib.parse import urlparse
from config import today
from cleaning.text_cleaner import clean_text

def validate_event(event, today_value=None):
"""Return (status, errors). Invalid records take precedence over expired status."""
today_value = today_value or today()
errors = []
if not clean_text(event.get("title")):
errors.append("Missing required field: title")
if not event.get("source_id"):
errors.append("Missing source_id; provide a stable source identifier")
start = event.get("start_date")
end = event.get("end_date")
if start and end and start > end:
errors.append("start_date must not be later than end_date")
url = event.get("registration_link")
if url:
parsed = urlparse(url)
if parsed.scheme not in {"http", "https"} or not parsed.hostname:
errors.append("registration_link must be a valid HTTP/HTTPS URL")
# The registration link is recommended, not required by default. Configure stricter rules as
needed.
if errors:
return "invalid", errors
if start and start < today_value.isoformat():
return "expired", []
return "upcoming", []
```

## module_2_3/validation/__init__.py

```
(Empty file)
```

## module_2_3/output/.gitkeep

File type: unknown — contents listed in the archive but not rendered in this PDF.

## module_2_3/.pytest_cache/README.md

```
# pytest cache directory #

This directory contains data from the pytest's cache plugin,
which provides the `--lf` and `--ff` options, as well as the `cache` fixture.

**Do not** commit this to version control.

See [the docs](https://docs.pytest.org/en/stable/how-to/cache.html) for more information.
```

## module_2_3/.pytest_cache/CACHEDIR.TAG

File type: .tag — contents listed in the archive but not rendered in this PDF.

## module_2_3/.pytest_cache/.gitignore

File type: unknown — contents listed in the archive but not rendered in this PDF.

## module_2_3/parsers/example_source_parser.py

```
"""Example source-specific parser template; adapt selectors to a real website."""
from bs4 import BeautifulSoup
from parsers.base_parser import BaseParser

class ExampleSourceParser(BaseParser):
"""Demonstrates data-* attributes as a stable source-specific contract."""
def parse(self, raw):
soup = BeautifulSoup(raw, "html.parser")
cards = soup.select("[data-event-card]")
records = []
for card in cards:
records.append({
"title": card.select_one("[data-title]").get_text(" ", strip=True) if
card.select_one("[data-title]") else "",
"organizer": card.get("data-organizer"),
"description": card.select_one("[data-description]").get_text(" ", strip=True) if
card.select_one("[data-description]") else "",
"start_date": card.get("data-start-date"),
"end_date": card.get("data-end-date"),
"location": card.get("data-location"),
"mode": card.get("data-mode"),
"registration_link": card.select_one("a[data-register]").get("href") if
card.select_one("a[data-register]") else None,
"prize": card.get("data-prize"),
"tags": [x.get_text(" ", strip=True) for x in card.select("[data-tag]")],
"source_id": card.get("data-source-id"),
})
return records
```

## module_2_3/parsers/json_parser.py

```
"""Parser for JSON event data."""
import json
from parsers.base_parser import BaseParser

_FIELD_ALIASES = {
"title": ("title", "name", "event_name", "eventTitle"),
"organizer": ("organizer", "organiser", "host", "organization"),
"description": ("description", "summary", "details"),
"start_date": ("start_date", "startDate", "start", "date"),
"end_date": ("end_date", "endDate", "end"),
"location": ("location", "venue", "city"),
"mode": ("mode", "event_mode", "format"),
```

```
"registration_link": ("registration_link", "registrationUrl", "register_url", "url"),
"prize": ("prize", "prize_pool", "reward"),
"tags": ("tags", "categories", "skills"),
"source_id": ("source_id", "id", "event_id"),
}

def _first(item, keys):
for key in keys:
if item.get(key) is not None:
return item[key]
return None

class JsonParser(BaseParser):
def parse(self, raw):
if isinstance(raw, str):
raw = json.loads(raw)
if isinstance(raw, dict):
# Support common wrappers, while treating a single event as a record.
for key in ("events", "data", "results", "items"):
if isinstance(raw.get(key), list):
raw = raw[key]
break
else:
raw = [raw]
if not isinstance(raw, list):
raise ValueError("JSON input must be an event object, list, or common list wrapper.")
output = []
for item in raw:
if not isinstance(item, dict):
output.append({"_parse_error": "Event record is not a JSON object"})
continue
output.append({field: _first(item, aliases) for field, aliases in _FIELD_ALIASES.items()})
return output
```

## module_2_3/parsers/generic_html_parser.py

```
"""Best-effort generic HTML parser. Site-specific selectors should override this."""
import json
import re
from bs4 import BeautifulSoup
from parsers.base_parser import BaseParser

class GenericHtmlParser(BaseParser):
def parse(self, raw):
if not isinstance(raw, str):
raise TypeError("HTML input must be a string.")
soup = BeautifulSoup(raw, "html.parser")
# Prefer JSON-LD Event data where present.
for script in soup.find_all("script", attrs={"type": "application/ld+json"}):
try:
data = json.loads(script.string or script.get_text())
candidates = data if isinstance(data, list) else [data]
for candidate in candidates:
if isinstance(candidate, dict) and candidate.get("@type") == "Event":
return [self._from_jsonld(candidate)]
except (json.JSONDecodeError, TypeError):
continue
title_tag = soup.find("h1") or soup.find("title")
title = title_tag.get_text(" ", strip=True) if title_tag else ""
description_tag = soup.find("meta", attrs={"name": "description"})
description = description_tag.get("content", "") if description_tag else ""
body = soup.get_text(" ", strip=True)
registration = None
for a in soup.find_all("a", href=True):
label = a.get_text(" ", strip=True).casefold()
if any(word in label for word in ("register", "registration", "apply", "join now")):
registration = a["href"]
break
# Simple labels such as "Organizer: Example Org" and "Prize: INR 50k".
def labelled(pattern):
match = re.search(pattern, body, re.IGNORECASE)
return match.group(1).strip() if match else None
start = labelled(r"(?:Start Date|Date)\\s*:\\s*([^|;]+?)(?=\\s+(?:End
Date|Location|Mode|Prize|Organizer)\\s*:|$)")
end = labelled(r"End Date\\s*:\\s*([^|;]+?)(?=\\s+(?:Location|Mode|Prize|Organizer)\\s*:|$)")
organizer = labelled(r"Organizer\\s*:\\s*([^|;]+?)(?=\\s+(?:Start Date|Date|End
Date|Location|Mode|Prize)\\s*:|$)")
location = labelled(r"Location\\s*:\\s*([^|;]+?)(?=\\s+(?:Start Date|Date|End
Date|Mode|Prize|Organizer)\\s*:|$)")
mode = labelled(r"Mode\\s*:\\s*([^|;]+?)(?=\\s+(?:Start Date|Date|End
Date|Location|Prize|Organizer)\\s*:|$)")
```

```
prize = labelled(r"Prize\\s*:\\s*([^|;]+?)(?=\\s+(?:Start Date|Date|End
Date|Location|Mode|Organizer)\\s*:|$)")
return [{
"title": title, "organizer": organizer, "description": description,
"start_date": start, "end_date": end, "location": location, "mode": mode,
"registration_link": registration, "prize": prize, "tags": [],
"source_id": None,
}]

@staticmethod
def _from_jsonld(item):
location = item.get("location")
if isinstance(location, dict):
location = location.get("name")
organizer = item.get("organizer")
if isinstance(organizer, dict):
organizer = organizer.get("name")
offers = item.get("offers")
if isinstance(offers, dict):
offers = offers.get("url")
return {
"title": item.get("name"),
"organizer": organizer,
"description": item.get("description"),
"start_date": item.get("startDate"),
"end_date": item.get("endDate"),
"location": location,
"mode": item.get("eventAttendanceMode"),
"registration_link": offers,
"prize": None,
"tags": item.get("keywords", []),
"source_id": item.get("identifier"),
}
```

## module_2_3/parsers/base_parser.py

```
"""Parser interface."""
from abc import ABC, abstractmethod

class BaseParser(ABC):
@abstractmethod
def parse(self, raw):
"""Return a list of dictionaries containing extracted event fields."""
raise NotImplementedError
```

## module_2_3/parsers/__init__.py

```
(Empty file)
```

## module_2_3/parsers/__pycache__/__init__.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/parsers/__pycache__/json_parser.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/parsers/__pycache__/generic_html_parser.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/parsers/__pycache__/base_parser.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/.pytest_cache/v/cache/nodeids

File type: unknown — contents listed in the archive but not rendered in this PDF.

## module_2_3/.pytest_cache/v/cache/lastfailed

File type: unknown — contents listed in the archive but not rendered in this PDF.

## module_2_3/validation/__pycache__/__init__.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/validation/__pycache__/validator.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/schema/__pycache__/__init__.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/schema/__pycache__/canonical_schema.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/cleaning/__pycache__/date_cleaner.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/cleaning/__pycache__/__init__.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/cleaning/__pycache__/location_cleaner.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/cleaning/__pycache__/text_cleaner.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/cleaning/__pycache__/url_cleaner.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/cleaning/__pycache__/prize_cleaner.cpython-313.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/tests/__pycache__/test_cleaning.cpython-313-pytest-9.0.2.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/tests/__pycache__/test_validation.cpython-313-pytest-9.0.2.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/tests/__pycache__/test_pipeline.cpython-313-pytest-9.0.2.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.

## module_2_3/tests/__pycache__/test_parsers.cpython-313-pytest-9.0.2.pyc

File type: .pyc — contents listed in the archive but not rendered in this PDF.